

美团前端一面

1. 获取页面元素位置与宽高的多种方法。
2. requestAnimationFrame原理，结合实际应用场景。
3. JS事件机制，结合一个复杂事件场景。
4. 实现一个bind方法，支持透传参数。
5. 实现Vue中的on,emit,off,once，支持自定义事件。
6. 实现一个双链表，支持增删改查。
7. Vue的双向绑定机制，结合源码分析。
8. 说出所有会引起浏览器重绘和重排的操作。
9. 页面性能监测方法，结合一个实际页面。
10. 使用原生JS实现一个轮播图组件。
11. 说出至少5种Web安全漏洞及其防御措施。
12. 简述同源策略及其限制，并给出几种常见的跨域解决方案。
13. 描述浏览器渲染页面的主要流程，并分析可能影响页面性能的因素。
14. 实现一个函数，将数字每三位用逗号分隔。
15. 分析如何使用媒体查询实现响应式布局。

1.获取页面元素位置与宽高

位置：通过元素的offsetLeft和offsetTop属性可以获取元素相对于其offsetParent元素的位置。

宽高：通过元素的offsetWidth和offsetHeight属性可以获取元素的宽高。

2.requestAnimationFrame原理

概念：requestAnimationFrame是一种由浏览器提供的动画API，可以在浏览器下一次重绘之前调用指定的函数来更新动画。

异步执行：requestAnimationFrame是异步执行的，浏览器会在下一次重绘之前调用传入的函数，而不会立即执行。

优化动画性能：使用requestAnimationFrame可以避免动画卡顿，优化动画性能。

3.js事件机制

事件流：JS事件机制包括事件捕获阶段、目标阶段和事件冒泡阶段。

传播顺序：在点击按钮时，事件首先在捕获阶段从根元素向下传播到目标元素，然后在目标阶段触发目标元素的监听事件，最后在冒泡阶段从目标元素向上传播回根元素。

4.js bind实现机制

函数方法：bind()是Function原型上的一个方法，用于绑定函数的this指向和参数。

实现思路：通过返回一个新函数，并在新函数中调用原函数，并传入指定的this和参数。

示例代码：

```
1
2  Function.prototype.bind = function(context, ...args) {
3    let self = this;
4    return function() {
5      return self.apply(context, args.concat(...arguments));
6    };
7  }
```

5.实现vue中的on,emit,off,once

定义：Vue中的事件总线用于组件之间的通信。

实现思路：

on(eventName, callback)：在事件中心添加事件监听。

emit(eventName, data)：触发事件中心的事件监听。

off(eventName, callback)：移除事件中心的监听。

once(eventName, callback)：添加一次性的事件监听。

示例代码：

```
1  export default {
2    install(Vue) {
3      Vue.prototype.$bus = new Vue();
4    };
5  };
```

6.js实现双链表

定义：双链表每个节点包含指向前一个节点和后一个节点的指针。

实现思路：通过创建Node类来表示节点，包含val、prev、next属性。链表类包含head、tail属性来表示头尾节点。

7.vue的双向绑定机制

原理：Vue通过响应式系统来实现双向绑定。在组件的data中定义的属性会被Vue转换为响应式属性，通过Object.defineProperty来实现对属性的getter和setter劫持。

示例：

```
1
2  let obj = {};
3  let internalVal = null;
4  Object.defineProperty(obj, 'val', {
5    get() {
6      return internalVal;
7    },
8    set(newVal) {
9      if (newVal === internalVal) return;
10     internalVal = newVal;
11     // 通知更新视图
12   }
13 })
```

8.浏览器重绘和重排

操作：以下操作会引起重绘或重排：

改变元素位置、大小等布局相关的属性

改变影响外部布局的属性，如float、position、overflow等

获取元素位置、大小相关的属性，如offsetWidth、getBoundingClientRect等

改变元素内容，如innerHTML等

改变元素样式，如style属性等

例外：使用transform属性可以避免重绘和重排。

页面性能监测

工具：使用Performance API进行页面性能监测，包括页面加载时间、DOM构建时间、资源加载时间等。